

Bizzy Books — Inventory Module

Implementation specification for retail-mode inventory tracking

Prepared for Claude Fable / Cowork · Author: Brad Caldwell · July 31, 2026 · Target user: Rachel (Gather and Glean / Garner Coffee)

1. Purpose and background

Bizzy Books is an existing iOS bookkeeping app (Swift, Firebase backend) built for a roofing contractor. Its current model assumes every purchase is immediately deductible: the user taps a category (Advertising, Supplies, Labor, etc.) and the amount reduces profit that same day. That model is correct for a service business with no goods held for sale.

The app is being extended for a retail user who sells physical merchandise — candles, perfume/cologne, apparel, coffee and tea — through a storefront plus a mobile trailer plus shelf space inside two other businesses. For that user, the contractor model produces a **wrong tax return**, because merchandise bought for resale is not an expense when purchased. It is an asset that only becomes deductible when it sells.

This document specifies the additions needed to make the app produce a correct return for an inventory business, plus a small amount of business insight that falls out of the same data for free. It is deliberately scoped *down* from a full point-of-sale system. Read section 3 (Design constraints) before writing any code — several obvious-seeming features are intentionally excluded, and building them would make the app worse for this user, not better.

2. The accounting concept the code must implement

Everything in this spec exists to serve one calculation, which is IRS Schedule C, Part III (Cost of Goods Sold):

```
Beginning inventory (cost of goods on hand Jan 1)
+ Purchases during the year (cost of goods bought for resale)
- Ending inventory (cost of goods on hand Dec 31)
= Cost of Goods Sold (COGS)
```

COGS is then subtracted from gross receipts to arrive at gross profit, and only *after* that do the ordinary expense lines (rent, advertising, supplies, etc.) come off. So the app now has two distinct kinds of money going out:

KIND	EXAMPLES	BEHAVIOR IN APP
Inventory (goods for resale)	Candles, dresses, sweaters, socks, perfume, retail bags of coffee beans and tea	Recorded immediately; posts to an asset bucket; does not reduce profit. Becomes deductible via COGS at year end.
Expense (consumed operating)	Ice, cups, lids, napkins, register tape, cleaning supplies, propane, rent, advertising, insurance, utilities, permit fees	Existing behavior. Reduces profit immediately.

THE KEY INSIGHT FOR THE DATA MODEL

A purchase is **always recorded the day it happens**. What changes is which bucket it lands in — asset now, deduction later. There is no scenario where the user delays entering something. "Record immediately, deduct on sale" is the whole rule.

2.1 How the app knows what sold — without recording sales

This is the central design decision and it must be understood before implementing anything else. There are two ways to compute COGS:

- Perpetual** — record every individual sale at the item level, decrementing counts in real time. Requires a POS flow, per-sale discipline, and typically barcode scanning. **Not being built.**
- Periodic** — never record sales. Record purchases only. Count what remains at year end. Whatever is missing must have sold. **This is what we are building.**

Under the periodic method, units sold is derived by subtraction, per item type:

```
units_sold = beginning_qty + purchased_qty_during_year - ending_counted_qty
```

This is why the year-end count screen is not an optional nicety — it is the mechanism that replaces sale recording entirely. No count, no COGS, no valid return.

3. Design constraints — what NOT to build

These exclusions are deliberate and were reasoned through against this specific user's operating pattern (she restocks roughly once or twice a year; her existing bookkeeping help is unreliable). Do not "helpfully" add these back in.

DO NOT BUILD	WHY
Point-of-sale / per-sale item recording	Requires perfect daily discipline. One missed sale silently corrupts every count. The periodic method needs none of it.
Barcode stickers on individual physical objects	Only pays off under perpetual tracking. Enormous labeling chore, zero benefit here.
Reorder points / low-stock alerts	Meaningless without live counts, and she does not make weekly reorder decisions.
Live inter-location transfer tracking	Would be required to report "what sold at which store," but demands recording every box moved. See §7.4 for what we do instead.
Per-item serial numbers or unique object IDs	We track item <i>types</i> with quantities, not individual objects.
Inventory tracking for the coffee bar	Beans, milk, syrups, cups, lids, and ice consumed into made-to-order drinks are treated as ordinary Supplies expense. Only Gather and Glean merchandise (and retail-packaged beans/tea sold sealed for home use) runs through inventory.
Payment processing of any kind	Out of scope entirely.

HARD RULE — NEVER VALUE INVENTORY AT STICKER PRICE

Ending inventory must be valued at **what she paid**, never at the marked retail price. This user prices with handwritten stickers and is highly prone to this error. Counting a candle that cost \$140 at its \$300 sticker overstates income by \$160 and causes her to overpay tax on money she never made. The UI must make cost-vs-price unambiguous at every entry point (see §6.3).

4. Mode switching

Add a business-type flag on the **user/company record**, not per transaction. Set during onboarding, changeable in Settings.

```
company: {
  businessType: "contracting" | "retail",    // default "contracting"
  ...
}
```

When `businessType == "retail"`:

- **Show:** Inventory category on Add Item; Inventory list view; Inventory Count screen; COGS block on the Tax Document screen.
- **Hide:** Projects, Estimates, Contracts, Warranties, Workers Comp records, and the gallons/fuel-record screen.

Existing contracting behavior must be entirely unchanged when the flag is `"contracting"`. This is an additive feature branch, not a rewrite.

5. Feature A — "Inventory" as a purchase category

5.1 Entry flow

On the existing **Add Item → Business** screen, add `Inventory` (goods for resale) to the category picker alongside Advertising, Supplies, Labor, etc.

Selecting it changes the form. Instead of a single amount field, the user is entering *a shipment*, which may contain multiple item types:

1. Standard header fields, unchanged: date, vendor, payment method, total amount, optional photo of the invoice.
2. A repeating **line item** section. Each line captures:
 - **Item type** — pick from existing list, or create new inline
 - **Quantity received** (integer)
 - **Unit cost** (what she paid per unit, excluding sales tax if she bought on resale certificate)
 - Auto-computed line extension = qty × unit cost
3. Running total of all lines displayed at the bottom. If it does not match the header total, show a soft warning but **allow the save** — freight, tax, and discounts will cause legitimate mismatches. Do not block her.

5.2 Creating a new item type inline

Tapping "+ New item type" opens a small sheet. Keep it to as few fields as possible — this is the friction point that determines whether she uses

the app at all.

FIELD	REQUIRED	NOTES
Display name	Yes	Free text, e.g. "Vanilla Essence Candle — Large". This is the only thing she ever sees or types.
Category	Yes	Picker: Candles / Fragrance / Apparel / Coffee & Tea / Other. Used to group the count screen and the reports. Keep the list short and user-editable.
Unit cost	Yes	Pre-filled from the line being entered.
Sale price	No	Sticker price. Optional. Used only for margin display — see §5.4 warning.
Internal ID	Auto	Generated UUID. Never displayed to the user. No visible SKU codes anywhere in the UI.

RESOLUTION IS THE USER'S CHOICE

She may have ~250 distinct item types, or she may prefer to lump ("Large Candles" as one type rather than 100 named scents). Both must work. More types = more insight, more typing. Do not enforce granularity, do not nag her to split types, and do not require every physical variation to be its own type.

5.3 The changing-cost problem — pick one rule and implement it

The same item type will be purchased at different unit costs over time (40 candles at \$140 in spring, 20 more at \$150 in fall). At year end, 9 remain and nobody — not her, not the app — can know which batch they came from. The app must therefore apply a consistent valuation rule.

Implement weighted moving average. On each receipt of an item type, recompute:

$$\text{new_avg_cost} = \frac{(\text{qty_on_hand} \times \text{current_avg_cost} + \text{qty_received} \times \text{receipt_unit_cost})}{\text{qty_on_hand} + \text{qty_received}}$$

Store this as `avgCost` on the item type. This is the value used to price ending inventory. Rationale: it is self-maintaining, needs no batch/lot tracking, is fully accepted by the IRS, and degrades gracefully if she enters things out of order. The alternative (most-recent-cost, i.e. FIFO-ish) is also acceptable but requires her to understand which cost is "current."

Also retain the full purchase history per item type so the rule can be recomputed or audited later. Store `lastCost` alongside `avgCost` for display.

CONSISTENCY IS A LEGAL REQUIREMENT, NOT A PREFERENCE

Whatever rule ships must be used every year thereafter. Do not offer a per-year toggle. If a settings switch exists at all, changing it should warn that switching valuation methods is an accounting method change.

5.4 Sale price handling

Sale price on the item type is optional and exists for one reason: showing her margin (sale price — avg cost) so she can see which products carry the store. It is **display only**.

NEVER LET SALE PRICE TOUCH A TAX FIGURE

Sale price must never be used to value inventory, compute COGS, or feed any number on the Tax Document screen. Enforce this at the data-flow level, not just by convention. Any code path where `salePrice` could reach the COGS calculation is a bug.

5.5 Inventory list view

A filtered view of everything categorized as Inventory. Implement either as a filter chip on the existing main feed or a dedicated tab — same underlying query.

Two toggleable presentations:

- **By purchase** — chronological receipts: date, vendor, total, expandable to lines. Mirrors the existing feed.
- **By item type** — one row per type: name, qty purchased this year, avg cost, last cost, optional sale price and margin.

The by-item-type view doubles as her reference sheet during the year-end count, so it should be reachable from the count screen.

6. Feature B — the year-end count ("Inventory Count")

A dated, saved, re-openable count session. Nominally December 31 but the date is user-set and it should tolerate being done a few days either side of year end — label it by tax year, not by date.

6.1 Locations

Locations are a user-managed list. Seed with these, all editable and deletable:

- Trailer
- Storefront
- Rostretta
- Columbus store presence
- Home / storage

Allow adding new locations at any time, including *mid-count* (she will remember the storage unit while standing in it). Each location: display name, optional note, active/archived flag.

LOCATIONS ARE A COUNTING CONVENIENCE, NOT A LIVE SYSTEM
Location is recorded **only** at count time, when she is physically standing in the place looking at the goods. There is no running per-location balance, no transfer recording, and no location field on purchases. This is what makes the feature nearly free to build and free for her to use.

6.2 Count flow

1. **Start count** — choose tax year, confirm date, see list of locations with progress indicators (not started / in progress / done).
2. **Pick a location** — she is standing in one place; she counts that place.
3. **Count screen for that location** — pre-populated list of every item type she purchased this year *plus* every type with a nonzero prior-year balance. She fills in quantity. Filling in blanks, not composing from scratch.
 - Grouped by category (Candles, Fragrance, Apparel, Coffee & Tea) with collapsible sections.
 - Blank ≠ zero. Distinguish "not counted here" from "counted, none here." Default state is blank; require an explicit 0 or a "none of this here" action.
 - Big tap targets and a number pad. She is doing this one-handed holding a clipboard or a phone in a cold storefront.
 - Search/filter by name for long lists.
 - "+ Add item type not on this list" — she will find things the app never saw (old trailer stock). Must be possible without leaving the count.
 - Running subtotal for the location at the bottom (qty and extended cost).
4. **Repeat per location.** Autosave continuously. She will be interrupted; she must be able to close the app mid-count and resume days later without losing anything.
5. **Review & finalize** — see §6.4.

6.3 Cost display during counting

Each line shows the unit cost the app will use (`avgCost`), clearly labeled **"cost you paid"** — never just "price," never "\$." Allow her to override a line's cost if she knows the app is wrong.

If sale price is also displayed anywhere on this screen, it must be visually subordinate and explicitly labeled "sticker price — not used for taxes." Given the risk described in §3, the safest implementation is to **omit sale price from the count screen entirely**.

6.4 Computation on finalize

For each item type, sum quantities across all locations. That total is the tax figure; the per-location breakdown is retained for her own use.

```
ending_qty[type]  = Σ counted_qty[type][location]  across all locations
ending_value[type] = ending_qty[type] × avgCost[type]

ENDING INVENTORY (Schedule C line 41) = Σ ending_value[type]

units_sold[type] = beginning_qty[type] + purchased_qty[type] - ending_qty[type]

COGS = beginning_inventory_value + purchases_value - ending_inventory_value
```

Present a review screen before finalizing that surfaces anything suspicious:

- Negative computed `units_sold` (counted more than she could have had — miscount, missed purchase, or an uncounted location).
- Item types purchased this year with zero counted anywhere (plausible if sold out, worth confirming).

- Locations left entirely uncounted.
- Item types with no cost on file.

These are warnings, not blockers. She can finalize anyway.

On finalize: lock the count (editable only via an explicit "reopen" action that logs the change), write ending inventory to the tax year, and **roll it forward as next year's beginning inventory automatically**. That roll-forward is essential and easy to forget.

6.5 Reports

Generate a PDF from a finalized count:

- **Summary page** — grand total ending inventory value, total units, breakdown by category, breakdown by location, and the Schedule C Part III block (beginning + purchases – ending = COGS).
- **One page per location** — every item type counted there: name, quantity, unit cost, extended cost, location subtotal.
- **Units-sold page** — per item type: beginning, purchased, ending, units sold. Aggregate only.

HONEST LABELING OF WHAT THE REPORT CAN CLAIM

The per-location pages are a **snapshot of where goods were sitting on the count date**. They are **not** sold-by-location data. Because stock moves between locations untracked, the app cannot know how many candles sold at the trailer versus the storefront. Never label anything "bestseller by location," "sales by store," or similar. Units sold is only valid in aggregate. Put a one-line footer on the location pages stating this.

7. Feature C — beginning inventory and onboarding

7.1 The problem

This user has been in business for years. Her beginning inventory is **not zero** — there is existing stock in the trailer, on two other stores' shelves, in the new storefront, and possibly at home. The app must not assume a zero opening balance.

7.2 Opening count flow

First-run for a retail user: an **Opening Inventory** flow that is mechanically the same screen as the year-end count, but seeds the catalog as it goes rather than working from a pre-populated list.

She walks each location and enters: item type (creating each one as she encounters it), quantity, unit cost. This single session simultaneously builds her item-type catalog and establishes her beginning inventory value. It is the same walk either way — she owes the tax side that number regardless.

Support: pausing and resuming indefinitely; adding locations mid-flow; and a manufacturer-barcode scan (VisionKit) purely to **auto-fill the item name** for factory-packaged goods and speed up cataloging. The scan creates the type; it is not for tracking sales. Handmade candles and unlabeled goods are hand-entered.

7.3 Missing cost data

She will not know what she paid for goods that have sat for years. Provide an explicit **estimated cost** path: mark the cost as an estimate, optionally derive it from sticker price × a margin assumption she sets once. Flag estimated-cost lines in the data and note them on the report. A good-faith reconstruction is acceptable when records are genuinely gone; it must be reasonable and consistently applied.

This price-archaeology cost is paid **once**. Everything purchased after onboarding carries a real invoice cost.

7.4 Manual beginning-balance entry

Also allow a plain numeric beginning-inventory entry per tax year, for the case where she or a preparer already has the figure and does not want to walk the store. This is the escape hatch — the return can be correct with just a number.

8. Tax Document screen additions

Add a Cost of Goods Sold block to the existing Tax Document screen, mapped to Schedule C:

LINE	SCHEDULE C	SOURCE
Gross receipts	Part I, line 1	Existing income total
Returns and allowances	Part I, line 2	New — optional entry
Beginning inventory	Part III, line 35	Prior year's ending, rolled forward
Purchases	Part III, line 36	Sum of Inventory-category purchases in year
Ending inventory	Part III, line 41	Finalized count total
Cost of goods sold	Part III, line 42	Computed
Gross profit	Part I, line 5	Receipts – returns – COGS
Expenses	Part II	Existing category totals, unchanged
Net profit	Part I, line 31	Existing, now net of COGS

Inventory purchases must be **excluded** from the Part II expense breakdown. If they appear in both places the return double-counts. Add a regression test for exactly this.

REQUIRED DISCLAIMER

Display in the app (Tax Document screen footer and on generated PDFs): this is a record-keeping tool, not tax advice, and figures should be reviewed by a qualified preparer. Non-negotiable — the app is being handed to a friend who self-files.

9. Data model

Firebase collections, scoped under the existing company/user document. Field names are suggestions; match existing app conventions.

```
itemTypes/{itemId}
  id      : string (uuid, never shown to user)
  name    : string      // "Vanilla Essence Candle – Large"
  category : string      // "Candles" | "Fragrance" | "Apparel" | "Coffee & Tea" | "Other"
  avgCost : number      // weighted moving average – used for valuation
  lastCost : number      // display only
  salePrice : number?    // optional, DISPLAY ONLY, never touches tax math
  costIsEstimate : bool  // true for reconstructed opening-stock costs
  qtyPurchasedYTD : number // per tax year, or derive from purchase lines
  active   : bool
  createdAt / updatedAt

inventoryPurchases/{purchaseId}
  date, vendor, paymentMethod, totalAmount, photoRef
  taxYear      : number
  lines        : [ { itemId, qty, unitCost, extension } ]
  // posts to ASSET bucket; must NOT appear in Part II expense totals

locations/{locationId}
  id, name, note, active
  // seeded: Trailer, Storefront, Rostretta, Columbus store presence, Home / storage

inventoryCounts/{countId}
  taxYear      : number
  countDate    : date
  type         : "opening" | "yearEnd"
  status       : "inProgress" | "finalized"
  finalizedAt  : timestamp?
  lines        : [ {
    itemId,
    locationId,
    qty,
    unitCostUsed,      // snapshot of avgCost at finalize – do not recompute later
    costOverridden : bool,
    extension
  } ]
  totals       : { byLocation: {...}, byCategory: {...}, grandTotal: number }

taxYears/{year}
  beginningInventory : number      // rolled forward from prior year's finalized count
  purchasesTotal     : number
  endingInventory    : number
  cogs               : number
```

SNAPSHOT COSTS AT FINALIZE

`unitCostUsed` must be frozen into the count line when finalized. If a later purchase changes `avgCost`, a previously filed year's ending inventory must not silently change. Historical returns have to stay reproducible.

10. Acceptance tests

SCENARIO	EXPECTED
Inventory purchase entered	Appears in feed and inventory list; net profit on Tax Document is unchanged ; Part II expense total unchanged.
Supplies purchase entered	Net profit decreases immediately. Existing behavior intact.
Two receipts of same type at different costs	<code>avgCost</code> = weighted average, not last cost, not simple mean. Verify: 40 @ \$140 then 20 @ \$150 → \$143.33.
Count entered across 3 locations	Tax ending inventory = sum of all three. Per-location breakdown retained and printable.
Units sold derivation	Beginning 10 + purchased 60 – ending 9 = 61 sold.
Count finalized	Ending inventory becomes next year's beginning inventory automatically.
Purchase added after finalize	Finalized year's ending inventory value does not change.
Sale price populated	No tax figure anywhere changes. Margin display only.
Count abandoned mid-session	Resumes with all entered data intact, correct location progress.
Contracting mode	No inventory UI visible anywhere; all existing screens and behavior identical to pre-change build.

11. Build order

1. **Mode flag** + show/hide plumbing (§4). Smallest change, unblocks everything.
2. **Item types** collection + inline creation sheet (§5.2).
3. **Inventory purchase category** with line items, posting to asset bucket (§5.1), plus weighted-average cost maintenance (§5.3).
4. **Tax Document COGS block** (§8) — even with a manually entered ending inventory, this alone makes the return correct. Ship-able checkpoint.
5. **Locations** collection with seeds (§6.1).
6. **Count screen** with per-location entry, autosave, finalize, roll-forward (§6.2–6.4).
7. **Opening inventory flow** (§7) — needed before she can use it for real, but mechanically a variant of #6.
8. **PDF reports** (§6.5).
9. *Optional later:* barcode-assisted cataloging, margin reporting, category-level insight.

Steps 1–4 produce a legally correct return on their own with a hand-entered ending inventory number. Everything after that is making the count easier and giving her business insight. Prioritize accordingly if time is short.